

Technical Problem Definition — Patent #5

Trust Assurance

Problem Statement (one sentence)

Computational trust continuously decays during operation unless actively maintained.

Trust infrastructure does not fail catastrophically. It rots. Entropy sources silently degrade. Quality scores drift downward. Trust chains grow deeper than policy allows. Anchoring latency creeps up. Source diversity narrows. Policies are overridden. None of these changes triggers an error. Each is individually small. Collectively, they transform a AAA-grade trust system into a LOW-grade one — without anyone noticing until an audit or a failure.

This is **Trust Decay**: the continuous, silent degradation of computational trust properties during normal operation.

Why This Problem Exists

Trust Decay is an emergent property of any running trust infrastructure. It occurs because:

1. Entropy sources are not static.

A system that starts with three independent, high-quality entropy sources may, over months, lose access to one (service discontinuation), see degraded quality from another (network

latency increasing staleness), and develop over-reliance on the remaining source. The system still produces trust artifacts. The artifacts still pass verification. But their quality has silently declined.

2. Operational pressure erodes policy.

Under production load, operators may increase rate limits, lower ECS thresholds, skip anchoring for "non-critical" requests, or allow deeper trust chains. Each override is individually rational. Collectively, they hollow out the trust guarantees the system was designed to provide.

3. Trust chains grow organically.

In multi-agent systems (Patent #4), trust chains emerge from agent delegation. Over time, chains that were designed for 3 levels of delegation may reach 8 or 10 levels. Each level of additional delegation may reduce trust quality. Without continuous enforcement, chains exceed their design limits.

4. Time decay is continuous.

Trust artifact freshness decays from the moment of issuance. A trust artifact that scored AAA at creation scores AA after 5 minutes and A after 10 minutes. If consumption patterns change (longer queuing times, larger pool buffers), average freshness decreases without any system error.

5. Regulatory requirements evolve.

A system compliant under GLI-19 v3.0 may not be compliant under v3.1. Regulatory updates occur outside the trust infrastructure's awareness. Without proactive adaptation, compliance silently lapses.

The defining characteristic of Trust Decay: It is not a failure. It is not an attack. It is not a misconfiguration. It is the natural thermodynamic tendency of a trust system to degrade over time under normal operation — like a battery losing charge, like entropy in a physical system. Without active maintenance, trust decays.

Why Existing Approaches Fail

Approach	Why it fails for Trust Decay
SIEM systems (Splunk, Elastic)	Monitor security events (intrusions, breaches). Trust Decay is not a security event — it's a quality degradation. SIEM has no concept of ECS scores, trust chain depth, or entropy source diversity.
APM tools (Datadog, New Relic)	Monitor application performance (latency, errors, throughput). Trust Decay is not a performance problem — the system is fast and error-free while trust quality silently degrades.
GRC platforms (ServiceNow, Archer)	Manage compliance workflows. Require manual evidence collection and periodic assessment. Trust Decay is continuous; periodic manual checks miss the gradual degradation between assessments.
Alerting systems (PagerDuty, OpsGenie)	Alert on threshold breaches. Trust Decay is sub-threshold — each individual metric may be within bounds while the aggregate trust posture has degraded significantly. It is the combination of multiple small drifts that constitutes decay.
Audit logs	Record what happened. Do not analyze whether what happened constitutes trust degradation. An audit log shows "source X was used." It does not show "source X has been used for 85% of all requests in the last 24 hours, exceeding the diversity policy."

The common failure: Existing tools monitor the symptoms of system health (errors, latency, security events). None monitors **trust health** — the continuous, multi-dimensional quality posture of a computational trust infrastructure.

Technical Effect (what solving this enables)

If Trust Decay is actively detected and counteracted:

1. **Trust quality is maintained over time.** Instead of silently degrading from AAA to B over months, the system detects quality decline and takes corrective action (rebalancing sources, enforcing anchoring, tightening policies) before trust drops below acceptable levels.
2. **Compliance is continuous, not periodic.** Instead of passing an annual audit and hoping nothing changed, the system continuously generates machine-readable compliance attestations that prove ongoing compliance at any moment. Compliance becomes a continuous property, not a periodic checkpoint.
3. **Trust infrastructure becomes self-healing.** When source diversity drops, the system automatically rebalances. When anchoring latency increases, the system automatically increases anchoring priority. When chain depth exceeds policy, the system blocks further delegation. The infrastructure actively maintains its own trust properties.
4. **Regulatory adaptation is proactive.** When a regulatory framework updates its requirements, the system compares current operations against new requirements, generates gap analysis, and identifies operations that would violate updated rules — before the new rules take effect.

Invention Thesis (one sentence)

A system for detecting, quantifying, and counteracting the continuous decay of computational trust during operation — through multi-dimensional trust health monitoring, anomaly detection against trust-specific metrics (not security or performance metrics), automated corrective response, continuous machine-readable compliance attestation via

pluggable regulatory framework adapters, and predictive trust quality analytics that forecast decay before it crosses critical thresholds.

Pressure Test

Question	Answer
Does this problem exist today?	Yes, but invisibly. Every running system with quality-dependent properties (not just trust) decays. Today's trust infrastructure is too new for decay to have been widely observed, but the physics of decay (source degradation, policy erosion, organic growth) are universal and inevitable.
Does this exist without AI?	Yes. Any long-running system that depends on external sources, quality thresholds, and policy compliance will decay: certificate infrastructure (cert expiration), network security (firewall rule accumulation), database performance (index bloat). Trust Decay is the same phenomenon in a new domain.
Is it a technical problem, not a business problem?	Yes. The problem is the absence of continuous multi-dimensional monitoring specific to trust quality metrics, with automated corrective response. The solution is a runtime engine that understands trust artifacts, trust chains, ECS scores, source provenance, and regulatory mappings — none of which exist in current monitoring systems.
Can it generate many implementations?	Yes. Trust health dashboards, regulatory adapters (EU AI Act, NIST, GLI-19, HIPAA, SOX, FedRAMP), predictive analytics, automated rebalancing, continuous

Question	Answer
	compliance attestation, multi-tenant governance, real-time drift detection — all are specific implementations of anti-decay.
Can you say it in one sentence?	"Computational trust continuously decays during operation unless actively maintained." ✓

The Battery Analogy

Trust Decay is to trust infrastructure what battery drain is to mobile devices:

- A fully charged battery (AAA trust) slowly loses charge during use.
 - The drain is not an error — it is physics.
 - Without monitoring, the user doesn't know until the device dies.
 - With monitoring (battery percentage), the user sees the decline in real time.
 - With active management (low-power mode, charging), the device maintains usable charge.
 - Trust infrastructure needs the same: a trust health meter, decay detection, and active maintenance.
-

This document defines the technical problem. RFC-0003 will specify the protocol. Patent #5 claims will naturally derive from the protocol.